COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010/Data Structures and Algorithm	**SECTION:**	CPE21S3
DATE PERFORMED:	August 18, 2026	**DATE SUBMITTED:**	August 23, 2026
NAME:	Gresola, Aryanna Keith G.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 6.1 **Searching Algorithm**

1. PROCEDURE OUTPUT

ILO A: Create C++ code for searching data using different techniques

Table 6-2a. Linear Search for Arrays

INPUT (LINEAR SEARCH - ARRAY)

```
5   template <typename T>
6   void linearSearchArr(int arrSize, T dataFind, T list[]) {
7
8       //Step 1: I := 0
9       int i = 0;
10      //Repeat while I <= n
11      while (i < arrSize) {
12          //  If (item = data[i])
13          if (dataFind == list[i]) {
14              //  Print "Searching is successful"
15              std::cout << "Searching is successful.\n";
16              return;
17          }
18          i++;
19      }
20      // Print "Searching is Unsuccessful"
21      std::cout << "Searching is unsuccessful." << std::endl;
22  }
```

HEADER

```
27      // testing
28      std::cout << "Array Linear Search result is: ";
29      linearSearchArr(max_size, userInput, dataset);
```

MAIN

OUTPUT

```
5 8 9 10 10 10 13 15 15 22 22 22 23 27 30 31 32 34 38 39 39 42 46 50 51 53 55
57 58 59 60 63 65 67 70 71 74 74 75 76 77 79 79 80 83 91 94 95 95 97
```

DATA GENERATED

```
What number to number to search? 6
Array Linear Search result is: Searching is unsuccessful.
```

```
1 2 7 7 8 10 13 13 15 15 17 18 19 19 26 29 32 34 34 36 37 38 40 40 42 43 44 45
 45 46 47 47 50 61 62 62 66 66 66 69 75 79 79 82 87 93 95 97 99 99
```

DATA GENERATED

```
What number to number to search? 2
Array Linear Search result is: Searching is successful.
```

OBSERVATION

The overall program performs a linear search to find the user input inside the array. The main function is done by linearSearchArr(max_size, userInput, dataset), which calls the linear search function. The important part of the code checks the elements one by one until the searched data is found and displays either "Searching is successful." or "Searching is unsuccessful." This shows how linear search goes through the array sequentially until it gets a result.

Table 6-2b. Linear Search for Linked List

INPUT (LINEAR SEARCH - LINKED LIST)

```
24  template <typename T>
25  void linearSearchLL(Node<T>* head, T dataFind){
26      while(head != nullptr){
27          if(head->data == dataFind){
28              std::cout << "Searching is successful." << std::endl;
29              return;
30          }
31          head = head->next;
32      }
33      std::cout << "Searching is unsuccessful." << std::endl;
34  }
```

HEADER

```
33      // create linked list: Y A N N A
34      Node<char>* name1 = new_node('Y');
35      Node<char>* name2 = new_node('A');
36      Node<char>* name3 = new_node('N');
37      Node<char>* name4 = new_node('N');
38      Node<char>* name5 = new_node('A');
39
40      name1->next = name2;
41      name2->next = name3;
42      name3->next = name4;
43      name4->next = name5;
44      name5->next = nullptr;
45
46      // linear
47      std::cout << "Linked List Linear Result is (Y): ";
48      linearSearchLL(name1, 'Y');
49      std::cout << "Linked List Linear Result is (X): ";
50      linearSearchLL(name2, 'X');
```

MAIN

OUTPUT

```
Linked List Linear Result is (Y): Searching is successful.
Linked List Linear Result is (X): Searching is unsuccessful.
```

OBSERVATION

The overall program performs a linear search on a linked list. The main function is done by linearSearchLL(name1, 'Y') and linearSearchLL(name2, 'X'), which search for the given character in the linked list. The syntax while(head != nullptr) keeps the search running through the nodes, while head = head->next moves to the next node. The condition if(head->data == dataFind) checks if the current node contains the searched data. The program clearly shows how pointers are used to move through a linked list during the search.

Table 6-3a. Binary Search for Arrays

INPUT (BINARY SEARCH - ARRAY)

```
68  template <typename T>
69  void binarySearchArr(int arrSize, T dataFind, T list[]){
70      int low = 0;
71      int up = arrSize - 1;
72
73      while(low <= up){
74          int mid = (low + up) / 2;
75          if(dataFind == list[mid]){
76              std::cout << "Search element is found!" << std::endl;
77              return;
78          }
79          else if(dataFind < list[mid]){
80              up = mid - 1;
81          }
82          else{
83              low = mid + 1;
84          }
85      }
86      std::cout << "Search element is not found." << std::endl;
87  }
```

HEADER

```
30      std::cout << "Array Binary Search result is: ";
31      binarySearchArr(max_size, userInput, dataset);
```

MAIN

OUTPUT

```
1 3 5 7 7 8 10 11 12 13 16 16 17 19 25 28 29 31 32 33 33 33 33 36 40
40 42 49 52 52 52 55 58 58 61 67 67 69 72 73 76 77 79 79 89 91 94 97
98 99
```

DATA GENERATED

```
What number to number to search? 7
Array Linear Search result is: Searching is successful.
Array Binary Search result is: Search element is found!
```

```
0 1 2 4 5 7 10 12 13 19 21 23 23 24 29 29 31 34 34 35 35 36 42 46 49
53 53 53 54 56 57 61 63 63 65 69 70 70 74 75 79 81 86 87 87 88 94 96
99 99
```

DATA GENERATED

```
What number to number to search? 3
Array Linear Search result is: Searching is unsuccessful.
Array Binary Search result is: Search element is not found.
```

OBSERVATION

The overall program performs a binary search to find the user input in the array. The syntax int low = 0, int up = arrSize - 1, and int mid = (low + up) / 2 are used to determine the search range and its middle element. The while(low <= up) continues the search, while if(dataFind == list[mid]) checks if the middle value is the target. The conditions else if(dataFind < list[mid]) and else update up or low to continue searching in the correct half. Compared to linear search, the program reduces the number of elements that need to be checked.

Table 6-3b. Binary Search for Linked List

INPUT (BINARY SEARCH - LINKED LIST)

```
48  template <typename T>
49  void binarySearchLL(Node<T>* start, T dataFind){
50      Node<T>* last = nullptr;
51      while(start != last){
52          Node<T>* middle = getMiddle(start, last);
53
54          if(middle->data == dataFind){
55              std::cout << "Searching is successful." << std::endl;
56              return;
57          }
58          else if(middle->data > dataFind){
59              last = middle;
60          }
61          else{
62              start = middle->next;
63          }
64      }
65      std::cout << "Searching is unsuccessful." << std::endl;
66  }
```

HEADER

```
52      // sorted linked list binary
53      Node<int>* node1 = new_node(5);
54      Node<int>* node2 = new_node(10);
55      Node<int>* node3 = new_node(15);
56      Node<int>* node4 = new_node(20);
57      Node<int>* node5 = new_node(25);
58
59      node1->next = node2;
60      node2->next = node3;
61      node3->next = node4;
62      node4->next = node5;
63      node5->next = nullptr;
```

```
69      // binary:
70      std::cout << "Linked List Binary Search result is: ";
71      binarySearchLL(node1, 5);
```

MAIN

OUTPUT

```
Linked List Binary Search result is: Searching is successful.
```

OBSERVATION

The overall program performs a binary search on a linked list using binarySearchLL(node1, 5). The syntax Node<T>* middle = getMiddle(start, last) is important because it finds the middle node before comparing it with the searched value. The condition if(middle->data == dataFind) checks if the value is found, while last = middle and start = middle->next change the search range depending on the result. The code works similarly to binary search in an array, but it needs to traverse the nodes to locate the middle.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1. Suppose you are doing a sequential search of the list [15, 18, 2, 19, 18, 0, 8, 14, 19, 14]. Utilizing both a linked list and an array approach to the list, use sequential search and identify how many comparisons would be necessary to find the key '18'?

INPUT

NODE.H:

Supple_HOA6p1 > C node.h > ...

```h
#ifndef NODE_H
#define NODE_H

template <typename T>
class Node{
public:
    T data;
    Node *next;
};

template <typename T>
Node<T> *newNode(T newData){
    Node<T> *newNode = new Node<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}

#endif
```

SEARCHINGALGO2.H:

Supple_HOA6p1 > C searchingAlgo2.h > linearSearchLLComp<T>(Node<T>*, T)

```cpp
#include <iostream>
#include "node.h"

#ifndef SEARCHINGALGO2_H
#define SEARCHINGALGO2_H

// problem 1:
template <typename T>
void linearSearchArrComp(int arrSize, T dataFind, T list[]){
    int i = 0;
    int comparisons = 0;

    while(i < arrSize){
        comparisons++;
        if (dataFind == list[i]){
            std::cout << "Searching successful" << std::endl;
            std::cout << "Number of comparisons: " << comparisons << std::endl;
        return;
        }
        i++;
    }
    std::cout << "Searching is unsuccessful." << std::endl;
}
```

```cpp

template <typename T>
void linearSearchLLComp(Node<T>* head, T dataFind){
    int comparisons = 0;
    while(head != nullptr){
        comparisons++;

        if(head->data == dataFind){
            std::cout << "Searching is successful." << std::endl;
            std::cout << "Number of comparisons: " << comparisons << std::endl;
            return;
        }
        head = head->next;
    }
    std::cout << "Searching is unsuccessful." << std::endl;
}

#endif
```

MAIN.CPP:

Supple_HOA6p1 > main.cpp > main()

```cpp
#include <iostream>
#include "node.h"
#include "searchingAlgo2.h"

const int max_size = 10;

int main(){
    int dataset[max_size] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    std::cout << "PROBLEM 1: " << std::endl;
    // testing array linear search:
    std::cout << "Array Linear Search result is: ";
    linearSearchArrComp(max_size, 18, dataset);

    // testing linked list linear search:
    // - creating linked list:
    Node<int>* node1 = newNode(15);
    Node<int>* node2 = newNode(18);
    Node<int>* node3 = newNode(2);
    Node<int>* node4 = newNode(19);
    Node<int>* node5 = newNode(18);
    Node<int>* node6 = newNode(0);
    Node<int>* node7 = newNode(8);
    Node<int>* node8 = newNode(14);
    Node<int>* node9 = newNode(19);
    Node<int>* node10 = newNode(14);

    node1->next = node2;
    node2->next = node3;
    node3->next = node4;
    node4->next = node5;
    node5->next = node6;
    node6->next = node7;
    node7->next = node8;
    node8->next = node9;
    node9->next = node10;
    node10->next = nullptr;

    // test:
    std::cout << "Linked List Linear Search result is: ";
    linearSearchLLComp(node1, 18);

    return 0;
}
```

OUTPUT

```
PROBLEM 1:
Array Linear Search result is: Searching successful
Number of comparisons: 2
Linked List Linear Search result is: Searching is successful.
Number of comparisons: 2
```

Problem 2. Modify your sequential search algorithm so that it returns the count of repeating instances for a given search element ‘k’. Test on the same list given in problem 1.

INPUT

NODE.H:

Supple_HOA6p1 > C node.h > ...

```
1  #ifndef NODE_H
2  #define NODE_H
3
4  template <typename T>
5  class Node{
6  public:
7      T data;
8      Node *next;
9  };
10
11 template <typename T>
12 Node<T> *newNode(T newData){
13     Node<T> *newNode = new Node<T>;
14     newNode->data = newData;
15     newNode->next = nullptr;
16     return newNode;
17 }
18
19 #endif
```

SEARCHINGALGO2.H:

```cpp
template <typename T>
int linearSearchArrCount(int arrSize, T dataFind, T list[]){
    int i = 0;
    int count = 0;

    while(i < arrSize){
        if(dataFind == list[i]){
            count++;
        }
        i++;
    }
    return count;
}

template <typename T>
int linearSearchLLCount(Node<T>* head, T dataFind){
    int count = 0;
    while(head != nullptr){
        if(head->data == dataFind){
            count++;
        }
        head = head->next;
    }
    return count;
}


#endif
```

MAIN.CPP:

Supple_HOA6p1 > main.cpp > main()

```cpp
#include <iostream>
#include "node.h"
#include "searchingAlgo2.h"

const int max_size = 10;

int main(){
    int dataset[max_size] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    int dataFind = 18;

    std::cout << "PROBLEM 2: " << std::endl;
    // testing array linear search:
    std::cout << "Array Repeat Count (18): "
    << linearSearchArrCount(max_size, dataFind, dataset) << std::endl;
```

```cpp
    // testing linked list linear search:
    // - creating linked list:
    Node<int>* node1 = newNode(15);
    Node<int>* node2 = newNode(18);
    Node<int>* node3 = newNode(2);
    Node<int>* node4 = newNode(19);
    Node<int>* node5 = newNode(18);
    Node<int>* node6 = newNode(0);
    Node<int>* node7 = newNode(8);
    Node<int>* node8 = newNode(14);
    Node<int>* node9 = newNode(19);
    Node<int>* node10 = newNode(14);

    node1->next = node2;
    node2->next = node3;
    node3->next = node4;
    node4->next = node5;
    node5->next = node6;
    node6->next = node7;
    node7->next = node8;
    node8->next = node9;
    node9->next = node10;
    node10->next = nullptr;

    // test:
    std::cout << "Linked List Repeat Count (18): "
    << linearSearchLLCount(node1, dataFind) << std::endl;

    return 0;
}
```

OUTPUT

```
PROBLEM 2:
Array Repeat Count (18): 2
Linked List Repeat Count (18): 2
```

Problem 3. Suppose you have the following sorted list [3, 5, 6, 8, 11, 12, 14, 15, 17, 18] and are using the binary search algorithm. If you wanted to find the key 8, draw a diagram that shows how the searching works per iteration of the algorithm. Prove that your drawing is correct by implementing the algorithm and showing a screenshot of the code and the output console.

INPUT

SEARCHINGALGO2.H:

```
68  // problem 3:
69  template <typename T>
70  void binarySearchArr(int arrSize, T dataFind, T list[]){
71      int low = 0;
72      int up =  arrSize - 1;
73      while(low <= up){
74          int mid = ( low + up ) / 2;
75          if(dataFind == list[mid]){
76              std::cout << "Search element is found!" << std::endl;
77              std::cout << "Found at index: " << mid << std::endl;
78              return;
79          }else if(dataFind < list[mid]){
80              up = mid - 1;
81          }else{
82              low = mid + 1;
83          }
84      }
85      std::cout << "Search element is not found." << std::endl;
86  }
87
88  #endif
```

MAIN.CPP:

Supple_HOA6p1 > main.cpp > ...

```cpp
#include <iostream>
#include "node.h"
#include "searchingAlgo2.h"

const int max_size = 10;

int main(){
    int dataset[max_size] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
    int dataFind = 8;

    std::cout << "PROBLEM 3: " << std::endl;
    std::cout << "Array Binary Search result is: ";
    binarySearchArr(max_size, dataFind, dataset);

    return 0;
}
```

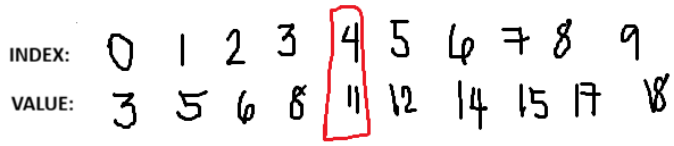
OUTPUT

```
PROBLEM 3:
Array Binary Search result is: Search element is found!
Found at index: 3
```

When the value being searched is not in the existing dataset:

```
PROBLEM 3:
Array Binary Search result is: Search element is not found.
```

DIAGRAM



ITERATION 1:

low = 0
up = 9

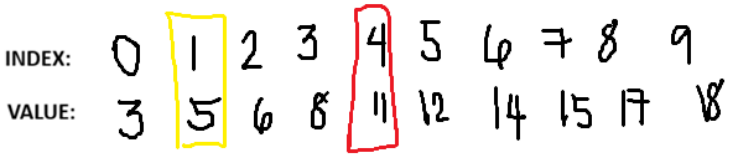
int mid = (low + up) / 2;

int mid = (0 + 9) / 2 = 4 // 11

8 < 11 since dataFind < list[mid]
the code will do this: up = mid - 1

NEW RANGE:

low = 0
up = 3



ITERATION 2:

low = 0
up = 3

int mid (low + up) / 2

0 + 3 / 2 = 1.5

8 < 5 since dataFind is greater the
code goes to: low = mid + 1

NEW RANGE:

low = 2
up = 3

INDEX:	0	1	2	3	4	5	6	7	8	9
VALUE:	3	5	6	8	11	12	14	15	17	18

ITERATION 3:
low = 2
up = 3

(low + up) / 2

2 + 3 / 2 = 2 // 6

since 8 is greater again than 6 the code will:
low = mid + 1

NEW RANGE:
low = 3
up = 3

INDEX:	0	1	2	3	4	5	6	7	8	9
VALUE:	3	5	6	8	11	12	14	15	17	18

ITERATION 4:
low = 3
up = 3

(low + up) / 2

(3 + 3) / 2 = 3 // 8

8 = 8 //

since 8 == 8, condition became true so the program stops.

Problem 4. Modify the binary search algorithm so that the algorithm becomes recursive. Using this new recursive binary search, implement a solution to the same problem for problem 3.

INPUT

MAIN.CPP:

Supple_HOA6p1 > main.cpp > ...

```cpp
#include <iostream>
#include "node.h"
#include "searchingAlgo2.h"

const int max_size = 10;

int main(){
    int dataset[max_size] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
    int dataFind = 8;

    std::cout << "PROBLEM 4: " << std::endl;
    std::cout << "Array Recursive Binary Search result is: ";
    recursiveBinarySearch(dataset, 0, max_size - 1, dataFind);

    return 0;
}
```

SEARCHINGALGO2.H:

```cpp
// problem 4:
template <typename T>
void recursiveBinarySearch(T list[], int low, int up, T dataFind){
    if(low > up){
        std::cout << "Search element is not found." << std::endl;
        return;
    }
    int mid = (low + up) / 2;
    if(dataFind == list[mid]){
        std::cout << "Search element is found." << std::endl;
        return;
    }else if(dataFind < list[mid]){
        recursiveBinarySearch(list, low, mid - 1, dataFind);
    }else{
        recursiveBinarySearch(list, mid + 1, up, dataFind);
    }
}
#endif
```

OUTPUT

```
PROBLEM 4:
Array Recursive Binary Search result is: Search element is found.
```

OBSERVATION

In this activity, I observed that the different search algorithms have the same goal, which is to find a specific data, but they work in different ways. Linear search checks the data one by one using list[i] for arrays and head->next for linked lists. Binary search, on the other hand, uses low, up, and mid to reduce the search range instead of checking every element.

For me, binary search is better when the data is already sorted because I think it is more efficient since it can reduce the number of elements that need to be checked. The recursive binary search also follows the same process, but instead of using a while loop, recursiveBinarySearch() calls itself with a smaller search range. This activity showed me that knowing different ways of implementing searching is important because each method can be more suitable depending on the data and how the program is designed.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Grammarly and ChatGPT

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Grammar correction and explanation of concepts.

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: Grammarly was used solely for grammar correction, while ChatGPT was used for explanation of concepts. These tools did not replace my original thinking, analysis, or authorship of the required work.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.